2026 ICSC Qualifications

Qualification Round Solutions

International Computer Science Competition (ICSC), Edition of 2026

Candidate Name:	Sourav Sharma
Submission Date:	June 21, 2026
Language Selection:	C++ (GCC 15.1)
Status:	Official Solutions Submission

Document compiled from candidate workspace
Local System: Arch Linux / EndeavourOS

1 Problem A: A History of Computing

The timeline below maps ten landmark events in computing and technology history plotted against the growth of transistor counts from 1936 to 2022.

Node	Year	Landmark Event	Significance in Computing History
1	1936	**Turing Machine Concept**	Alan Turing publishes his paper *"On Computable Numbers..."*, formalizing the Turing Machine and setting the theoretical foundation for computer science.
2	1947	**Invention of the Transistor**	John Bardeen, Walter Brattain, and William Shockley invent the point-contact transistor at Bell Labs, replacing vacuum tubes.
3	1958	**First Integrated Circuit**	Jack Kilby (TI) and Robert Noyce (Fairchild) independently invent the integrated circuit, allowing components to reside on one chip.
4	1969	**ARPANET & Unix Launch**	First packet transmission on ARPANET. Concurrently, the Unix operating system is created at Bell Labs, shaping modern OS design.
5	1981	**Introduction of the IBM PC**	IBM releases the Model 5150, standardizing personal computing architecture and establishing the dominance of the PC and MS-DOS.
6	1991	**Public Release of the WWW**	Tim Berners-Lee introduces the World Wide Web, creating HTTP and HTML. Concurrently, Linus Torvalds releases the Linux kernel.
7	2000	**Y2K Bug & Gigabit Barrier**	Turn of the millennium is successfully managed. Intel releases the Pentium 4, pushing processor speeds past the 1 GHz barrier.
8	2007	**Introduction of the iPhone**	Apple launches the iPhone, combining multi-touch interfaces, mobile apps, and web access, ushering in the mobile computing era.
9	2012	**Deep Learning Breakthrough**	AlexNet wins the ImageNet competition, demonstrating the power of deep convolutional neural networks on GPUs, starting the AI boom.
10	2022	**ChatGPT & Generative AI**	OpenAI releases ChatGPT, bringing Large Language Models (LLMs) and generative artificial intelligence into mainstream public use.

Table 1: Landmark events in computing history (1936 to 2022).

2 Problem B: Pixel Art

2.1 Mathematical Formulations and Approach

1. **Checkerboard Pattern (`shape = "checkerboard"`)**
   To alternate between filled (1) and empty (0) cells such that the top left cell is always 0, we use the coordinate sum parity:

   $$\text{grid}[i][j] = (i + j) \pmod 2$$

   For the top left cell $(0, 0)$, this evaluates to $(0+0) \pmod 2 = 0$, satisfying the requirement.

2. **Centered Diamond Pattern (`shape = "diamond"`)**
   For an odd grid size N, the center cell is located at coordinate (c, c) where $c = \lfloor N/2 \rfloor$. The diamond shape consists of all cells whose Manhattan (L_1) distance to the center is at most c:

   $$|i - c| + |j - c| \leq c$$

   Cells satisfying this inequality are set to 1; others are set to 0. This creates an exact centered rhombus.

2.2 Source Code Implementation (C++)

The complete implementation of the Pixel Art grid generator in C++ is shown below:

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <cmath>
#include <stdexcept>

std::vector<std::vector<int>> generate_shape(int n, std::string shape) {
    std::vector<std::vector<int>> grid(n, std::vector<int>(n, 0));

    if (shape == "checkerboard") {
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {

                grid[i][j] = (i + j) % 2;
            }
        }
    } else if (shape == "diamond") {
        if (n % 2 == 0) {
            throw std::invalid_argument("Grid size N must be odd for the diamond shape.");
        }
        int center = n / 2;
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {

                if (std::abs(i - center) + std::abs(j - center) <= center) {
                    grid[i][j] = 1;
                } else {
                    grid[i][j] = 0;
                }
            }
        }
    } else {
        throw std::invalid_argument("Unknown shape. Only 'checkerboard' and 'diamond' are supported.");
```

```cpp
    }

    return grid;
}

int main() {
    try {
        std::string n_str, shape;
        if (!std::getline(std::cin, n_str)) return 0;
        if (!std::getline(std::cin, shape)) return 0;

        int n = std::stoi(n_str);

        std::vector<std::vector<int>> grid = generate_shape(n, shape);

        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                std::cout << grid[i][j] << (j == n - 1 ? "" : " ");
            }
            std::cout << "\n";
        }
    } catch (const std::invalid_argument& e) {
        std::cerr << "Input Error or Validation Failed: " << e.what() << std
    ::endl;
        return 1;
    } catch (const std::exception& e) {
        std::cerr << "An unexpected error occurred: " << e.what() << std::
    endl;
        return 1;
    }

    return 0;
}
```

3 Problem C: Moore's Law

3.1 Transistor Count Prediction for 2026

We are given the reference year 1971 with $T_0 = 2,300$ transistors (Intel 4004). The elapsed time to 2026 is $t = 2026 - 1971 = 55$ years. Using Moore's formula:

$$T(t) = T_0 \times 2^{t/2}$$

Substituting our parameters:

$$T(55) = 2,300 \times 2^{55/2} = 2,300 \times 2^{27.5}$$

First, we compute the exponential term:

$$2^{27.5} = 2^{27} \times \sqrt{2} = 134,217,728 \times 1.41421356237 \approx 189,812,531.025$$

Then, we multiply by the reference count:

$$T(55) \approx 2,300 \times 189,812,531.025 \approx 436,568,821,357.5$$

Predicted Transistor Count (Moore's Law)

Using Moore's Law formulation, the predicted transistor count for a commercial microprocessor in the year 2026 is:

$$T(55) \approx \mathbf{436,568,821,358} \text{ transistors (approx. 436.57 Billion)}$$

3.2 Comparison to Actual 2026 Processors and Moore's Law Analysis

Let us examine state of the art processors from 2025/2026 such as the Apple M4/M5 series:

- **Base Apple M4/M5:** Contains **28 Billion** transistors.
- **Apple M4 Max:** Contains **92.5 Billion** transistors.
- **Apple M4 Ultra:** Contains **185 Billion** transistors.

Discussion:
The 1971 formula's prediction of **436.57 Billion** is an overestimate. It exceeds the base M4/M5 by **15.6**×, the high-end M4 Max by **4.7**×, and the dual die M4 Ultra by **2.36**×.

This gap demonstrates that while chip capacity has increased exponentially, the doubling rate has slowed down. In the last decade, physical limits of silicon (e.g., atomic scaling limits, quantum tunneling, and thermal density/"Dark Silicon") have made pure physical transistor scaling much harder. Modern performance gains are instead sustained via multi chiplet modules, custom accelerators (NPUs/GPUs), and 3D stacking.

3.3 Fitting the Doubling Period

We can compute the exact doubling period d in years that fits the historical growth from the Intel 4004 to today. Rearranging $T_{\text{actual}} = T_0 \times 2^{55/d}$:

$$d = \frac{55}{\log_2(T_{\text{actual}}/T_0)}$$

1. **For the Base Apple M4/M5 (28 Billion):**

$$\frac{T_{\text{actual}}}{T_0} = \frac{28,000,000,000}{2,300} \approx 12,173,913 \implies \log_2(12,173,913) \approx 23.538$$

$$d = \frac{55}{23.538} \approx \mathbf{2.34} \text{ years}$$

2. **For the High-End Apple M4 Max (92.5 Billion):**

$$\frac{T_{\text{actual}}}{T_0} = \frac{92,500,000,000}{2,300} \approx 40,217,391 \implies \log_2(40,217,391) \approx 25.261$$

$$d = \frac{55}{25.261} \approx \mathbf{2.18} \text{ years}$$

Thus, the historical growth rate over 55 years fits a doubling period of approximately **2.18 to 2.34 years**, remarkably close to Gordon Moore's predicted 2-year cycle.

4 Problem D: The Simultaneous Strike

4.1 Bug Analysis in the Replay Pseudocode

The bug in the provided pseudocode lies in line 3 and 4: the loop checks for a KO condition (`player1_hp <= 0 OR player2_hp <= 0`) **between individual attack events** rather than at the **end of a frame**.

When a simultaneous strike occurs (e.g. frame 512 where both players land attacks that would reduce both of their HP below 0), the stable sort orders one event before the other. The first event is processed, reducing that player's opponent's HP to 0. In the next iteration, the check in line 3 immediately triggers, breaking out of the loop **before** applying the second player's attack event. This ignores the second player's simultaneous strike, preventing a double KO. The corrected logic must process all events occurring in the same frame before performing the KO check.

4.2 Source Code Implementation (C++)

The corrected implementation of the game replay processor in C++ is shown below:

```cpp
#include <iostream>
#include <vector>
#include <tuple>
#include <algorithm>
#include <stdexcept>
#include <sstream>

std::vector<int> processGame(std::vector<std::tuple<int, int, int>> events, int H) {
    int hp1 = H;
    int hp2 = H;

    std::sort(events.begin(), events.end(), [](const std::tuple<int, int, int>& a, const std::tuple<int, int, int>& b) {
        return std::get<1>(a) < std::get<1>(b);
    });

    size_t i = 0;
    size_t n = events.size();

    while (i < n) {

        if (hp1 <= 0 || hp2 <= 0) {
            break;
        }

        int current_frame = std::get<1>(events[i]);

        int damage_to_1 = 0;
        int damage_to_2 = 0;

        while (i < n && std::get<1>(events[i]) == current_frame) {
            int attacker = std::get<0>(events[i]);
            int damage = std::get<2>(events[i]);

            if (attacker == 1) {
                damage_to_2 += damage;
            } else if (attacker == 2) {
                damage_to_1 += damage;
```

```
            } else {
                throw std::invalid_argument("Player number must be either 1 or 2.");
            }
            i++;
        }

        hp1 -= damage_to_1;
        hp2 -= damage_to_2;
    }

    return {std::max(0, hp1), std::max(0, hp2)};
}

int main() {
    try {
        std::string hp_line;
        if (!std::getline(std::cin, hp_line)) return 0;
        int H = std::stoi(hp_line);

        std::vector<std::tuple<int, int, int>> events;
        std::string line;
        while (std::getline(std::cin, line)) {
            line.erase(0, line.find_first_not_of(" \t\r\n"));
            if (line.empty()) continue;

            std::stringstream ss(line);
            int player, frame, attack;
            if (ss >> player >> frame >> attack) {
                events.push_back(std::make_tuple(player, frame, attack));
            }
        }

        std::vector<int> final_hp = processGame(events, H);

        std::cout << final_hp[0] << " " << final_hp[1] << std::endl;

    } catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << std::endl;
        return 1;
    }
    return 0;
}
```

5 Problem E: PageRank in the Age of AI Slop

5.1 PageRank Foundations

- **Random-Surfer Interpretation:** PageRank models a surfer browsing the web. At each step, the surfer either clicks an outgoing link on the current page with probability d, or teleports to a completely random page in the network with probability $1 - d$. The PageRank represents the stationary probability that the surfer lands on a given page.
- **Role of Damping Factor d:** The damping factor (typically $d = 0.85$) represents the link-following probability. It guarantees that the PageRank transition matrix is irreducible and aperiodic, ensuring convergence to a unique stationary distribution and preventing the surfer from getting trapped in web sinks/dead-ends.
- **Recursive Equation:** A page's importance is defined by the importance of the pages that link to it. Since a page's PageRank is computed using the PageRanks of its predecessor pages, the equation is recursive.
- **Practical Solution:** It is solved using the **Power Iteration** method. The system is written in matrix form: $R^{(t+1)} = \frac{1-d}{N}\mathbf{1} + dP^T R^{(t)}$, and multiplied iteratively until the vector converges (the L_1 distance between steps is below a small threshold).

5.2 Derivation of S_A (Total PageRank of AI Pages)

Let the web consist of N total pages, split into h human pages (set H) and a adversarial AI pages (set A). Let $S_A = \sum_{p \in A} PR(p)$ and $S_H = \sum_{p \in H} PR(p)$. Since PageRanks sum to 1, we have $S_H = 1 - S_A$. Let $\alpha = a/N$ be the fraction of AI pages.

Summing the standard PageRank equation over all pages $p_i \in A$:

$$S_A = \sum_{p_i \in A} PR(p_i) = \sum_{p_i \in A} \left(\frac{1-d}{N} + d \sum_{p_j \to p_i} \frac{PR(p_j)}{L(p_j)} \right)$$

$$S_A = a\frac{1-d}{N} + d \sum_{p_i \in A} \sum_{p_j \to p_i} \frac{PR(p_j)}{L(p_j)}$$

Substituting $a/N = \alpha$ and changing the order of summation on the double sum:

$$S_A = \alpha(1-d) + d \sum_{p_j} \frac{PR(p_j)}{L(p_j)} L(p_j \to A)$$

where $L(p_j \to A)$ is the number of links from p_j that point to the AI set A. We split the sum into human and AI source pages:

1. **For AI pages ($p_j \in A$):** AI pages link only to other AI pages. Thus, $L(p_j \to A) = L(p_j)$. The term becomes:
$$\sum_{p_j \in A} \frac{PR(p_j)}{L(p_j)} L(p_j) = \sum_{p_j \in A} PR(p_j) = S_A$$
2. **For human pages ($p_j \in H$):** Human pages link to k pages chosen uniformly at random from all N pages. On average, the fraction of links pointing to A is $a/N = \alpha$. Thus, $L(p_j \to A) = \alpha L(p_j)$. The term becomes:
$$\sum_{p_j \in H} \frac{PR(p_j)}{L(p_j)} (\alpha L(p_j)) = \alpha \sum_{p_j \in H} PR(p_j) = \alpha S_H$$

Substituting these back into the equation for S_A:

$$S_A = \alpha(1-d) + d(S_A + \alpha S_H)$$

Since $S_H = 1 - S_A$, we substitute it in:

$$S_A = \alpha(1-d) + dS_A + d\alpha(1-S_A)$$

$$S_A = \alpha - d\alpha + dS_A + d\alpha - d\alpha S_A$$

$$S_A = \alpha + d(1-\alpha)S_A$$

$$S_A(1 - d(1-\alpha)) = \alpha$$

Closed-Form Derivation for S_A

The total PageRank of all AI pages S_A is defined by:

$$S_A = \frac{\alpha}{1 - d(1-\alpha)}$$

5.3 Tipping Fraction and Vulnerability Discussion

The tipping fraction α^* is the fraction at which the AI pages collectively hold as much PageRank as human pages ($S_A = S_H = 0.5$):

$$\frac{\alpha^*}{1 - d(1-\alpha^*)} = \frac{1}{2} \implies 2\alpha^* = 1 - d(1-\alpha^*) = 1 - d + d\alpha^*$$

$$2\alpha^* - d\alpha^* = 1 - d \implies \alpha^*(2-d) = 1 - d$$

$$\alpha^* = \frac{1-d}{2-d}$$

At the standard damping factor $d = 0.85$:

$$\alpha^* = \frac{1-0.85}{2-0.85} = \frac{0.15}{1.15} = \frac{3}{23} \approx 0.13043 \quad (13.04\%)$$

PageRank Tipping Point & Vulnerability Analysis

For a standard damping factor $d = 0.85$, the critical tipping fraction α^* is:

$$\alpha^* \approx \mathbf{13.04}\%$$

Vulnerability Analysis:
If adversarial AI generated spam makes up just **13.04%** of the web, it siphons and traps **50%** of the entire web's PageRank. This demonstrates that PageRank is highly vulnerable to link farms and Sybil attacks. Since human pages link outward randomly, they leak PageRank to the AI network. Because AI pages form a closed group and never link back to human pages, they act as a spider trap, accumulating siphoned authority.